

✦ DESIGN PATTERNS ✦

Final Exam Study & Memorization Guide

SE 506 / CSE 2215 — Design Patterns
Institute of Information Technology, University of Dhaka

🔗 OOP Foundations

🔗 Refactoring & Code Smells

📁 Creational Patterns

📁 Structural Patterns

📁 Behavioral Patterns

📄 Cheat Sheet

Built from 11 real past question papers (2013–2024), with every unique question answered in plain, easy language — aligned with **refactoring.guru**'s explanations of design patterns and Fowler's refactoring catalog. Duplicate questions are merged, with how many times each one appeared in past papers noted next to it — so you know exactly what's worth prioritizing.

Sources: BSSE 2nd Yr 4th Sem Final 2024 (CSE 2215) · BSSE 3rd Yr 5th Sem Final 2024 (SE 506) · 5th Sem Finals 2013, 2014, 2017 · Midterm 2016 · Final 2020 (SE 506)

Table of Contents

1	OOP Foundations	Q1 – Q9
2	Refactoring & Code Smells — Theory	Q10 – Q27
3	Code Smell Identification & Refactoring Exercises	Q28 – Q35
4	Creational Patterns	Q36 – Q48
5	Structural Patterns	Q49 – Q55, Q65
6	Behavioral Patterns	Q56 – Q63, Q66
7	Memory Tricks 🧠	mnemonics
8	Final Cheat Sheet 📄	quick lookup tables

SECTION 1: OOP Foundations

Q1. What are the three main categories of design patterns? Briefly describe the purpose of each. *(asked 2x)*

A: Design patterns fall into three groups, based on **what problem they solve**:

- **Creational patterns** – deal with **object creation**. They hide the messy details of "how" and "when" objects are made, so your code doesn't get locked into one specific class. Examples: Singleton, Factory Method, Abstract Factory, Builder, Prototype.
- **Structural patterns** – deal with **how classes and objects are put together** to form bigger structures, while keeping things flexible. Examples: Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight.
- **Behavioral patterns** – deal with **how objects talk to each other and share responsibility**. They're about communication and assigning who does what. Examples: Observer, Strategy, Command, Template Method, Chain of Responsibility, Mediator, State, Iterator.

Simple way to remember: **Creational = birth, Structural = body, Behavioral = brain.**

Q2. Why is access to non-static variables not allowed from static methods in OOP? *(asked 2x)*

A: A **static method** belongs to the **class itself**, not to any particular object. It can run even before a single object has been created (e.g., `Main()`). A **non-static (instance) variable** only exists once an object is created — it lives inside that object's memory. Since a static method has no guarantee that any object exists yet, there is no "instance" to pull that variable from. Allowing it would mean asking "whose variable?" with no answer — so the compiler blocks it.

Q3. What does encapsulation mean in OOP? Why is it considered a vital concept? *(asked 2x)*

A: **Encapsulation** means bundling data (fields) and the methods that operate on that data into a single unit (a class), and **restricting direct access** to the internal state from outside — usually by making fields `private` and exposing controlled access through public methods (getters/setters).

Why it matters:

- **Protects data integrity** — outside code can't set a field to an invalid value directly.
- **Hides implementation details** — you can change how something works internally without breaking code that uses it.
- **Reduces coupling** — other classes depend only on the public interface, not on internal structure.

Q4. What are the basic features of OOP? Which one is the most important, and why?

A: The four pillars are:

1. **Encapsulation** – bundling and hiding data.
2. **Abstraction** – showing only essential details, hiding complexity.
3. **Inheritance** – reusing behavior from a parent class.
4. **Polymorphism** – same interface, different behaviors.

If you must pick "the most important," most textbooks (and Refactoring Guru's OOP intro) point to **Abstraction**, because it's the foundation everything else is built on — you can't sensibly encapsulate, inherit, or polymorph without first deciding what the *essential* interface of a concept is. (Some argue Encapsulation, since it's what makes safe reuse possible — either answer is acceptable if justified.)

Q5. What does object inheritance and object composition mean? Advantages/disadvantages of each for reusability?

A:

- **Inheritance ("is-a")**: A subclass extends a parent class and reuses its code automatically.
- ✓ Easy to reuse code quickly; supports polymorphism naturally.
- ✗ **Tight coupling** to the parent — a change in the parent can break children. Fixed at compile time. Can lead to fragile, deep hierarchies (Refactoring Guru calls this "inheritance breaks encapsulation").
- **Composition ("has-a")**: A class contains an instance of another class and delegates work to it.
- ✓ **Loose coupling** — you can swap the contained object at runtime. Favors flexibility.
- ✗ Slightly more code to write (you must write delegating methods).

GoF principle (and Refactoring Guru repeats this constantly): **"Favor composition over inheritance."**

Q6. Can we replace an interface with an abstract class? Justify your answer. *(asked 3x)*

A: Partially — it depends on the language and the need.

- An **abstract class** can declare abstract methods (like an interface) **and** hold shared state/fields **and** provide default method implementations.
- But in single-inheritance languages (Java, C#), a class can only **extend one** abstract class, while it can **implement many interfaces**. So if a class needs to fulfill multiple contracts, an abstract class cannot fully replace an interface.
- Also, interfaces express a pure **capability/contract** ("can do X"), while abstract classes express **shared identity/is-a relationship**. Using an abstract class where multiple unrelated classes need the same capability would force an artificial inheritance relationship.

Conclusion: No, not fully — interfaces support multiple inheritance of type and pure contracts; abstract classes don't.

Q7. What is the philosophical difference between an abstract class and an interface? Give example scenarios.

A: | | Abstract Class | Interface | |---|---|---| | Represents | "What an object **is**" | "What an object **can do**" | | Can hold state? | Yes | No (traditionally) | | Can have implemented methods? | Yes (partial implementation) | Historically no (default methods in newer Java allowed, but not the point) | | Inheritance | Single | Multiple |

Example: `Bird` (abstract class) has shared state like `age`, `weight`, and a common method `eat()`. But `Flyable` (interface) is a capability — an `Airplane` and a `Bird` are unrelated by identity but both `Flyable`. Use an abstract class when subclasses share **identity and common code**; use an interface when unrelated classes share a **capability**.

Q8. "Talk to the interface, not to the concrete class" — explain this statement with appropriate reasoning.

A: This means your code should depend on **abstractions (interfaces/abstract types)**, not on specific concrete implementations. Instead of:

```
ArrayList<String> list = new ArrayList<>();
```

you write:

```
List<String> list = new ArrayList<>();
```

Why: This follows the **Dependency Inversion Principle**. If your code only knows about the interface `List`, you can swap `ArrayList` for `LinkedList` later without touching any code that uses `list`. It reduces coupling, makes testing easier (you can inject mocks), and is the backbone idea behind almost every design pattern (Strategy, Factory, Observer all rely on "coding to an interface").

Q9. Any kind of software development is possible with other programming paradigms — why do we need Object-Oriented Programming in the first place?

A: It's true — anything computable can be written in procedural or functional style too. OOP isn't about making things *possible*, it's about making large systems **maintainable**:

- **Real-world modeling** — objects map naturally to real entities (Customer, Order), making the design intuitive.
- **Encapsulation** reduces accidental interference between parts of a large codebase.
- **Polymorphism & Inheritance** let you extend behavior without rewriting existing code (Open/Closed Principle).
- **Design patterns** (this whole course!) are only expressible cleanly because OOP gives us objects, interfaces, and inheritance to build flexible, reusable structures.

In short: OOP trades a bit of upfront structure for long-term **flexibility, reusability, and easier collaboration** in large systems.

SECTION 2: Refactoring & Code Smells — Theory

Q10. What is Refactoring? Why do we refactor our code? *(asked 2x)*

A: Refactoring is the process of changing the **internal structure** of code to make it cleaner, without changing its **external behavior** (the program still does the same thing from the outside).

We refactor because:

- It **improves code readability and maintainability**.
- It makes bugs easier to find and fix.
- It makes the code easier to extend with new features.
- It reduces **technical debt** that builds up over time.

Refactoring Guru's core rule: *refactor in small, safe steps, run tests after each step.*

Q11. What are the advices we must keep in mind before refactoring? Write a brief explanation.

A:

1. **Have tests first.** Before you touch the code, make sure you have a reliable test suite so you can confirm behavior didn't change.
2. **Refactor in small steps.** Don't try to fix everything at once — do one small transformation, verify, then move to the next.
3. **Don't mix refactoring with adding features.** Keep "cleaning" commits separate from "new behavior" commits.
4. **Know when to stop.** Refactor until the code is clear enough — don't over-engineer or gold-plate.
5. **Communicate with your team**, especially with shared code, so merges don't conflict badly.

Q12. Define code smell. Name 10 different common code smells. *(asked 2x)*

A: A **code smell** is a surface-level indicator in the source code that usually points to a **deeper problem** in the design. It's not a bug — the code still works — but it signals the code is harder to understand, extend, or maintain than it should be.

10 common code smells (Refactoring Guru's catalog groups them into Bloaters, OO Abusers, Change Preventers, Dispensables, Couplers):

1. Long Method
2. Large Class
3. Long Parameter List
4. Duplicate Code
5. Switch Statements (Type Code)
6. Feature Envy
7. Data Clumps
8. Primitive Obsession
9. Shotgun Surgery
10. Message Chains

(Others worth knowing: Speculative Generality, Refused Bequest, Lazy Class, Comments, Parallel Inheritance Hierarchies, Middle Man, Divergent Change.)

Q13. What is "Feature Envy"? What are the remedies of this code smell? *(asked 3x)*

A: Feature Envy happens when a method in Class A is more interested in the data of Class B than its own — it repeatedly calls getters on another object to do its work. This is a sign the method is **in the wrong class**.

```
// Feature Envy example
class Invoice {
    double calculateTotal(Customer c) {
        return c.getPrice() * c.getQuantity() * (1 - c.getDiscount());
    }
}
```

This method barely touches `Invoice`'s own data — it "envies" `Customer`'s data.

Remedies:

- **Move Method** — move the method to the class whose data it uses most.
 - **Extract Method** — if only part of the method envies another class, split that part out and move just that piece.
-

Q14. "A method is long when it has more than 20 lines of code" — comment on this. When is a method considered long? What are the remedies of long method? *(asked 2x)*

A: This statement is **not strictly true**. Line count is a rough heuristic, not a hard rule. A method can be:

- **Short but unclear** (using cryptic one-liners), or
- **Longer but perfectly readable** if it's a simple, linear sequence of clearly named steps.

A method is better judged "long" when:

- It does **more than one thing** (violates Single Responsibility).
- It requires scrolling to read.
- It needs a comment to explain each block (a sign it should be its own method).

Remedies (Refactoring Guru — Long Method techniques):

- **Extract Method** — pull a block of code into its own well-named method.
 - **Replace Temp with Query** — replace a local variable with a method call.
 - **Decompose Conditional** — split complex if/else logic into smaller methods.
 - **Replace Method with Method Object** — turn a huge method into its own class.
 - **Replace Conditional with Polymorphism** — if the method has type-based branching.
-

Q15. "Comments are often used as deodorant" — justify this statement. What are the remedies of comment? *(asked 4x — the most repeated question in the whole archive!)*

A: This means comments are often sprayed on top of confusing code to "mask the smell" instead of actually fixing the underlying problem — just like deodorant hides body odor without addressing the cause. A comment like:

```
// check if customer is eligible AND has not exceeded limit AND account is active
if (c.getAge() > 18 && c.getBalance() < c.getLimit() && c.status == 1) { ... }
```

is a comment explaining messy code — instead the code should be **rewritten to explain itself**.

Remedies:

- **Extract Method** and give the method a descriptive name that replaces the need for the comment — e.g., `isEligibleForLoan(customer)`.
- **Rename Variable/Method** so intent is obvious without explanation.
- **Introduce Assertion** if the comment was documenting an assumption.
- Keep comments **only** for things code truly cannot express: *why* a weird workaround exists, legal notes, or complex business/domain justifications — never to explain *what* the code does.

Q16. What do you understand by semantic duplication? Give one example of such duplication.

A: Semantic duplication is when two pieces of code look *textually different* but do the **exact same logical thing**. Regular duplicate-code detectors (that check for identical text) miss this, because the wording is different even though the *meaning/behavior* is repeated.

Example:

```
// Method A
double getDiscount(double price) {
    return price * 0.10;
}

// Method B (different wording, same job)
double applyTenPercentOff(double amount) {
    return amount - (amount * 0.90 == amount ? 0 : amount * 0.10);
}
```

Both compute a 10% discount — they are semantically the same, just written differently.

Q17. What does Parameter Object mean? Give one example of a Parameter Object.

A: Parameter Object is a refactoring technique where you replace a **group of parameters that always travel together** with a single object that wraps them.

Before:

```
void createEvent(int day, int month, int year, int startHour, int endHour) { ... }
```

After (Parameter Object):

```
class DateRange {
    Date date;
    int startHour, endHour;
}

void createEvent(DateRange range) { ... }
```

This is the fix for the **Long Parameter List** and **Data Clumps** code smells.

Q18. What are the different levels of duplication? Give one example for each level. *(asked 2x)*

A: Duplication can appear at different scopes:

1. **Line-level duplication** — the exact same line repeated within a method. - e.g., `total += price * qty;` appearing twice in the same loop unnecessarily.
2. **Block/Method-level duplication** — a chunk of logic (several lines) copy-pasted across methods. - e.g., `GetCsv()` and `GetCsvWithHeader()` both loop through persons and build the exact same "Name,Email,Age" string line — only the header differs.
3. **Class-level duplication** — two whole classes with almost the same structure and behavior. - e.g., `SalesReportGenerator` and `PurchaseReportGenerator` doing nearly identical work with different field names.
4. **Algorithm-level (semantic) duplication** — different code, same underlying logic (see Q16).

Fix at every level: Extract Method → Pull Up Method → Extract Superclass / Template Method, depending on scope.

Q19. What is Refused Bequest? Give one example of Refused Bequest and refactor it. *(asked multiple times)*

A: Refused Bequest happens when a subclass inherits from a parent class but **doesn't want or use most of the inherited methods/fields** — it "refuses" the inheritance gift. This is a sign the inheritance relationship is wrong.

Example (smell):

```
class Bird {
    void fly() { ... }
}

class Penguin extends Bird {
    // Penguin can't fly! Overrides fly() to throw an exception
    void fly() { throw new UnsupportedOperationException(); }
}
```

Refactor: Use **Push Down Method** (move `fly()` out of `Bird` into a `FlyingBird` subclass) or **Replace Inheritance with Delegation/Composition**:

```
class Bird { }

class FlyingBird extends Bird {
    void fly() { ... }
}

class Penguin extends Bird { } // no forced fly()
```

Q20. What is Primitive Obsession? Give one example of primitive obsession and refactor it. What are the remedies?

(asked 4x)

A: Primitive Obsession is over-using basic/primitive types (`int` , `String` , `double`) to represent domain concepts that deserve their own small class.

Example (smell):

```
class Order {
    String currencyCode; // 0, 1...
    double amount;
}
```

Using a raw `String` / `double` pair for money means validation logic (currency conversion rules, formatting) gets scattered everywhere `amount` is used.

Refactor (Replace Data Value with Object):

```
class Money {
    private final double amount;
    private final String currencyCode;
    Money add(Money other) { ... } // validation lives here, once
}
class Order {
    Money total;
}
```

Remedies: Replace Data Value with Object, Replace Type Code with Class, Introduce Parameter Object (when several primitives travel together), **Extract Class**.

Q21. What is a Lazy Class? Give an example of a lazy class. How can a lazy class be refactored? (asked 2x)

A: A Lazy Class is a class that doesn't do enough to justify its existence — maybe it used to do more, or was created for future use that never materialized. It just adds overhead (extra file, extra indirection) without real value.

Example:

```
class PersonName {
    String first, last;
    String getFull() { return first + " " + last; }
}
```

If this class is barely used anywhere, and its one method could easily live inside `Person` , it's "lazy."

Refactor:

- **Inline Class** — fold its fields/methods back into the class that uses it.
- **Collapse Hierarchy** — if it's a subclass adding almost nothing over its parent, merge them.

Q22. What is Oddball Solution? Give one example.

A: An Oddball Solution is when the **same problem is solved in a different, inconsistent way** somewhere else in the codebase — instead of reusing an existing, proven solution, a new/different one is invented for what is essentially the same situation.

Example: Your codebase already has a generic `Cache<K,V>` class used everywhere for caching. But in one new module, a developer writes a brand-new hand-rolled `HashMap` + manual expiry-check logic to do caching again, instead of reusing `Cache<K,V>` . That's an "oddball" — a one-off, inconsistent solution to an already-solved problem.

Fix: Recognize the duplicate intent and **replace the oddball code with the established/standard solution** (essentially "Substitute Algorithm" toward the existing shared component).

Q23. Which of the following can be considered as refactoring? Only yes/no answer will do. *(asked 2x)*

i. We'll just refactor the code to support logging. — **No** ii. Can you refactor the code so that it authenticates against LDAP instead of Database? — **No** iii. We have too much duplicate code, we need to refactor the code to eliminate duplication. — **Yes** iv. This class is too big, we need to refactor it. — **Yes** v. We need to implement caching to improve performance. — **No**

Rule of thumb: If it **adds new behavior/feature** (logging, LDAP auth, caching = new functionality), it is **NOT** refactoring. If it only **restructures existing code without changing what it does** (removing duplication, splitting a big class), it **IS** refactoring.

Q24. What is the relation between design patterns and refactoring? *(asked 2x)*

A: They are two sides of the same coin, working at different times:

- **Refactoring** improves *existing* code's structure without changing behavior — it's often the **process**.
- **Design patterns** are proven, reusable **target structures/solutions** you refactor *toward*.

In practice: you spot a code smell (e.g., a big switch on type codes), and you **refactor** the code by applying a **design pattern** as the destination (e.g., "Replace Conditional with Polymorphism" turns messy code into the **Strategy** or **State** pattern). Many refactoring techniques literally end with "...and now you have pattern X." So: **refactoring is the journey; design patterns are common, well-known destinations.**

Q25. What do you understand by "Collapse Hierarchy"? Give an example. Which code smells can be removed using this method? *(asked 3x)*

A: Collapse Hierarchy is a refactoring technique used when a **subclass and its superclass have become too similar** — there's barely any real difference between them anymore, so keeping them separate just adds unnecessary complexity. You **merge the subclass and superclass into one class**.

Example:

```
class Employee {
    double salary;
}

class Manager extends Employee {
    // Manager used to have extra fields, but they were all removed over time
}
```

Since `Manager` adds nothing anymore, merge it:

```
class Employee {
    double salary;
    boolean isManager; // if needed
}
```

Removes: Lazy Class and **Speculative Generality** (when a hierarchy was built "just in case" but never actually needed).

Q26. What does Extract Method mean? Which code smells are removed using this process? *(asked 2x)*

A: Extract Method takes a fragment of code that can be grouped together and turns it into its **own method** with a name that explains its purpose, then calls that new method from the original location.

Before:

```
void printOwing() {
    printBanner();
    // print details
    System.out.println("name: " + name);
    System.out.println("amount: " + getOutstanding());
}
```

After:

```
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}

void printDetails(double outstanding) {
    System.out.println("name: " + name);
    System.out.println("amount: " + outstanding);
}
```

Removes: Long Method, Duplicate Code (once extracted, the method can be reused elsewhere), and reduces the need for **Comments** (the method name replaces the comment).

Q27. Write down the probable causes leading to Long Method and Long Class code smells.

A: Causes of Long Method:

- Developers keep **adding "just one more line"** to an existing method instead of creating a new one (path of least resistance).
- Fear of creating too many small methods ("too much indirection").
- Copy-pasting logic instead of reusing/refactoring.
- Trying to handle multiple responsibilities in one place.

Causes of Long Class (God Class):

- A class keeps absorbing new responsibilities over time (violates Single Responsibility Principle).
- Poor initial design — everything was dumped into one "manager"/"utility" class.
- Fear of breaking things by splitting a class that "already works."
- Lack of code reviews to catch growing complexity early.

SECTION 3: Code Smell Identification & Refactoring Exercises (from real exam code)

Q28. The `Employee` class with `_type` field and a `switch` in `PayAmount()` (`ENGINEER`/`SALESMAN`/`MANAGER`) — what smells do you see? Refactor the code.

A: Smells found:

- **Primitive Obsession** — using an `int _type` with magic constants (`ENGINEER = 0`) instead of a real type.
- **Switch Statements / Type Code** — the `switch(_type)` smell; every time a new employee type is added, this switch must be edited (violates Open/Closed Principle).
- Risk of **Shotgun Surgery** — if another method also switches on `_type` elsewhere, you'd have to update multiple places.

Refactor — Replace Type Code with Subclasses + Replace Conditional with Polymorphism:

```
abstract class Employee {
    protected double monthlySalary;
    abstract double payAmount();
}

class Engineer extends Employee {
    double payAmount() { return monthlySalary; }
}

class Salesman extends Employee {
    double commission;
    double payAmount() { return monthlySalary + commission; }
}

class Manager extends Employee {
    double bonus;
    double payAmount() { return monthlySalary + bonus; }
}
```

Now adding a new employee type means adding a new subclass — no existing code is touched (**Open/Closed Principle** satisfied). This is essentially the **Strategy/State**-style structure the Refactoring Guru catalog points to when eliminating switch-on-type-code.

Q29. The Validator / ValidatorFactory code — `if(type == "string" || type == "int" || ...)` — what smells, and refactor. *(the most-repeated code exercise — appeared 4 times!)*

A: Smells found:

- **Switch Statements (if-chain on type code)** — the long `if/||` chain checking string types.
- **Primitive Obsession** — `type` is a raw `String` used to mean "which validator to build."
- Adding a new supported type means **editing existing code** (violates Open/Closed Principle).

Refactor idea 1 — using a Collection (as hinted by the handwritten exam note in the paper!):

```
class ValidatorFactory {
    private static final List<String> SUPPORTED_TYPES =
        Arrays.asList("string", "int", "double", "float");

    public Validator createValidator(String type) {
        if (SUPPORTED_TYPES.contains(type)) {
            return new Validator();
        }
        return null;
    }
}
```

This removes the repetitive `||` chain — a form of **Replace Conditional with a lookup / Consolidate Conditional Expression**.

Refactor idea 2 — full OOP fix (Factory Method + Polymorphism), better for extensibility:

```
interface Validator { boolean isValid(Object obj); }

class StringValidator implements Validator {
    public boolean isValid(Object obj) { /* string-specific check */ return true; }
}

class NumberValidator implements Validator {
    public boolean isValid(Object obj) { /* int/double/float check */ return true; }
}

class ValidatorFactory {
    private static final Map<String, Supplier<Validator>> registry = new HashMap<>();
    static {
        registry.put("string", StringValidator::new);
        registry.put("int", NumberValidator::new);
        registry.put("double", NumberValidator::new);
        registry.put("float", NumberValidator::new);
    }
    public Validator createValidator(String type) {
        Supplier<Validator> maker = registry.get(type);
        return maker == null ? null : maker.get();
    }
}
```

This is a genuine **Factory Method pattern** — new types are added by registering, not by editing `if` conditions.

Q30. CsvGenerator / Person with GetCsvWithHeader , GetCsv , GetCsvWithHeaderAndFooter — identify smells and refactor. (asked 2x)

A: Smells found:

- **Duplicate Code** — all three methods loop through `persons` and build `"Name,Email,Age\n"` rows identically; only the header/footer differs.
- **Speculative Generality** possibility if `GetCsv()` (no header) is never actually used anywhere.

Refactor (Extract Method + Template Method style):

```
class CsvGenerator {  
    private String buildRows(List<Person> persons) {  
        StringBuilder str = new StringBuilder();  
        for (Person p : persons) {  
            str.append(p.getName()).append(",")  
                .append(p.getEmail()).append(",")  
                .append(p.getAge()).append("\n");  
        }  
        return str.toString();  
    }  
  
    public String getCsv(List<Person> persons) {  
        return buildRows(persons);  
    }  
  
    public String getCsvWithHeader(List<Person> persons) {  
        return "Name,Email,Age\n" + buildRows(persons);  
    }  
  
    public String getCsvWithHeaderAndFooter(List<Person> persons) {  
        return "Name,Email,Age\n" + buildRows(persons)  
            + "Total no of persons: " + persons.size();  
    }  
}
```

The duplicated loop is pulled into one shared `buildRows()` — a textbook **Extract Method** fix for **Duplicate Code**.

Q31. String foundPerson(String[] people) with repeated if (people[i].equals("Don")) return "Don"; chains — identify smells and refactor.

A: Smells found:

- **Duplicate Code** — the same `if (...equals(x)) return x;` pattern repeated for "Don", "John", "Kent".
- Adding a new name means copy-pasting another `if` block.

Refactor:

```
String foundPerson(String[] people) {  
    List<String> candidates = Arrays.asList("Don", "John", "Kent");  
    for (String person : people) {  
        if (candidates.contains(person)) {  
            return person;  
        }  
    }  
    return "";  
}
```

This is **Consolidate Duplicate Conditional Fragments** — the repeated pattern becomes a single loop against a list.

Q32. toStringHelper(StringBuffer result) (XML tag rendering with recursive children) — find the smells, name the refactoring process, refactor.

A: Smells found:

- **Long Method** — it does several unrelated jobs in one method: opening tag, writing value, looping children, closing tag.
- Slight **Feature Envy** on `TagNode`'s own `children()` / `attributes` — reasonable here since it's the node's own data, but the method mixes levels of abstraction (string building + recursion + conditionals all mixed together).

Refactoring process: Extract Method (also called **Compose Method** — break one big method into several well-named smaller steps).

Refactor:

```
private String toStringHelper(StringBuffer result) {
    appendOpeningTag(result);
    appendValueIfPresent(result);
    appendChildren(result);
    appendClosingTag(result);
    return result.toString();
}

private void appendOpeningTag(StringBuffer result) {
    result.append("<").append(name).append(attributes.toString()).append(">");
}

private void appendValueIfPresent(StringBuffer result) {
    if (!value.equals("")) result.append(value);
}

private void appendChildren(StringBuffer result) {
    Iterator it = children().iterator();
    while (it.hasNext()) {
        TagNode node = (TagNode) it.next();
        node.toStringHelper(result);
    }
}

private void appendClosingTag(StringBuffer result) {
    result.append("</").append(name).append(">");
}
```

Now `toStringHelper` reads like a table of contents — each line is a clear step.

Q33. `Customer customer = site.getCustomer(); if (customer == null) plan = BillingPlan.basic(); else plan = customer.getPlan();` — refactor to remove the null checking. *(asked 2x)*

A: Smell: Repeated **null-checking** scattered through the code (**Refactoring Guru → Introduce Null Object**). Every caller of `getCustomer()` has to remember to check for null.

Refactor — Introduce Null Object pattern:

```
abstract class Customer {
    abstract BillingPlan getPlan();
    abstract boolean isNull();
}

class RealCustomer extends Customer {
    private BillingPlan plan;
    BillingPlan getPlan() { return plan; }
    boolean isNull() { return false; }
}

class NullCustomer extends Customer {
    BillingPlan getPlan() { return BillingPlan.basic(); }
    boolean isNull() { return true; }
}

class Site {
    Customer getCustomer() {
        return customer != null ? customer : new NullCustomer();
    }
}
```

Client code becomes:

```
Customer customer = site.getCustomer();
BillingPlan plan = customer.getPlan(); // no null check needed anywhere!
```

This removes conditional logic entirely from every calling site — the "do nothing / default" behavior is baked into `NullCustomer`.

Q34. findBooks() / validateTitle() / validateAuthor() — identify code smells with brief explanation and refactor.

A: Smells found:

- **Long Method** — `findBooks` mixes validation, looping, and two different filtering conditions in one method.
- **Nested Conditionals / Arrow Code** — the `if` inside `for` inside `if` creates deep nesting that's hard to read.
- **Duplicate Code** — the two validators (`validateTitle` , `validateAuthor`) both check "empty" and "contains newline" almost identically, just worded differently (semantic duplication, see Q16).
- Possible **bug-smell**: inside the `else if` , it compares `book.getTitle().contentEquals(author)` — likely should be `book.getAuthor()` , a sign the logic itself is confusing because it's too tangled to read clearly (this is exactly why code smells matter — they hide real bugs!).

Refactor (Extract Method + Decompose Conditional + remove duplication):

```
public List<Book> findBooks(String title, String author, LocalDate firstDate, LocalDate endDate) {
    List<Book> foundBooks = new ArrayList<>();
    if (!isValid(title) || !isValid(author)) return foundBooks;

    for (Book book : books) {
        if (isWithinDateRange(book, firstDate, endDate) || matchesTitleAndAuthor(book, title, author)) {
            foundBooks.add(book);
        }
    }
    return foundBooks;
}

private boolean isValid(String field) {
    return field != null && !field.isEmpty() && !field.contains("\n");
}

private boolean isWithinDateRange(Book book, LocalDate firstDate, LocalDate endDate) {
    return book.getPublishedOn().isAfter(firstDate) && book.getPublishedOn().isBefore(endDate);
}

private boolean matchesTitleAndAuthor(Book book, String title, String author) {
    return book.getTitle().contentEquals(title) && book.getAuthor().contentEquals(author); // bug fixed
}
```

`validateTitle` and `validateAuthor` collapse into one shared `isValid()` helper — removing both the duplication and the deep nesting.

Q35. TextEditor / UserInterface with hard-coded editor.save() / editor.print() direct calls — explain why this tight coupling is a problem. What would happen if you wanted to add "undo"?

A: Problem: `UserInterface` is **directly and tightly coupled** to `TextEditor` — it calls `editor.save()` and `editor.print()` directly. This means:

- `UserInterface` must know the exact method names of `TextEditor` .
- You **cannot** treat "save" and "print" as interchangeable, storable, or queueable *actions* — they're just fixed method calls.
- If you wanted to add an **"undo"** feature, you'd have no clean way to know **what action was just performed** or **how to reverse it** — you'd have to add messy tracking flags/history variables directly into `UserInterface` , further tangling the two classes together.

The fix (leads into Command pattern, see Q42).

SECTION 4: Creational Patterns

Q36. What are the benefits of using the Singleton pattern? Explain in brief.

A: Singleton ensures a class has **only one instance** and gives a **global access point** to it.

Benefits:

- **Controlled access to a single instance** — useful for things that must not be duplicated (a config manager, a connection pool, a logger).
- **Saves memory** — only one object is ever created, instead of many redundant ones.
- **Global point of access** — any part of the program can reach it without passing it around everywhere.
- **Lazy initialization possible** — the instance can be created only when first needed, saving startup resources.

(Watch out — Refactoring Guru also flags Singleton's downsides: it can hide dependencies, make unit testing harder, and act like a global variable in disguise. Mentioning both sides shows deeper understanding.)

Q37. Which design pattern provides a way for object cloning? Explain a scenario where this pattern can be used. *(asked 2x)*

A: The Prototype pattern.

How it works: Instead of creating a new object via `new` and manually copying all fields, the object provides a `clone()` method that copies itself, including its current state.

Scenario: Imagine a graphics editor where a user draws a complex `Shape` object (with many nested properties: color, position, texture, sub-shapes). Instead of rebuilding it from scratch and re-configuring every property to make an identical copy, you simply call `shape.clone()` to get an exact, independent duplicate — much faster and avoids re-doing complex setup.

```
interface Prototype { Prototype clone(); }
class Shape implements Prototype {
    Color color; Point position;
    public Shape clone() {
        Shape copy = new Shape();
        copy.color = this.color;
        copy.position = new Point(this.position); // deep copy of mutable fields
        return copy;
    }
}
```

Q38. Which design pattern helps reduce memory requirement (memory optimization)? How does this memory optimization actually work? What other pattern automatically comes with this implementation? *(asked 2x)*

A: The Flyweight pattern.

How it works: Flyweight splits an object's data into:

- **Intrinsic state** — data that is **shared and identical** across many objects (e.g., a tree's texture/shape data). Stored **once** in a shared "flyweight" object.
- **Extrinsic state** — data that is **unique per instance** (e.g., a tree's x,y position). Passed in from outside (by the client) each time it's needed, not stored in the flyweight.

By sharing the heavy, repeated (intrinsic) data across thousands of objects instead of duplicating it in every single instance, memory usage drops dramatically.

Pattern that automatically comes with it: The **Factory pattern** (specifically a "Flyweight Factory") — because you need a controlled way to look up an **existing shared flyweight** or create a new one if it doesn't exist yet, ensuring flyweights are properly reused instead of duplicated.

Q39. What is the Flyweight design pattern? Explain its purpose and how it helps improve performance and memory efficiency. Discuss intrinsic/extrinsic state with examples, and typical scenarios where it's most beneficial.

A: *(Builds on Q38 — same pattern, fuller answer for the higher-mark version of this question.)*

Purpose: To support **huge numbers of fine-grained objects efficiently** by sharing common data instead of storing it redundantly in each object.

Intrinsic state example: In a forest-rendering game, thousands of `Tree` objects share the same mesh, texture, and color per species — this is **intrinsic** and stored once inside a shared `TreeType` flyweight object.

Extrinsic state example: Each individual tree's **x, y coordinate** on the map is unique — this is **extrinsic**, kept outside the flyweight and supplied by the client (a `Forest` class) when rendering.

```
class TreeType { // Flyweight - intrinsic state
    String name, color, texture;
    void draw(int x, int y) { /* uses x,y (extrinsic) passed in */ }
}

class TreeFactory {
    static Map<String, TreeType> cache = new HashMap<>();
    static TreeType getTreeType(String name, String color, String texture) {
        String key = name + color + texture;
        return cache.computeIfAbsent(key, k -> new TreeType(name, color, texture));
    }
}

class Tree { // extrinsic state holder
    int x, y;
    TreeType type;
    void draw() { type.draw(x, y); }
}
```

Typical scenarios: Text editors (sharing glyph/character formatting objects), game particle systems, GUI icon caching, map/tile-based games — anywhere you have **massive numbers of similar objects**.

Q40. Compare and contrast Factory Pattern and Abstract Factory Pattern with appropriate examples. In which scenario should each be used?

A: | | Factory Method | Abstract Factory | |---|---|---| | Purpose | Creates **one product**, lets subclasses decide which concrete class to instantiate | Creates **families of related products** that must work together | | Structure | One method, overridden in subclasses | An interface with multiple creation methods, one per product type | | Example | `DialogBox.createButton()` — different OS dialogs create different button styles | `GUIFactory` that creates a **matching** `Button` + `Checkbox` + `Scrollbar` for a whole theme (Windows vs Mac) |

Factory Method example:

```
abstract class Dialog {
    abstract Button createButton(); // subclasses decide the concrete type
}
class WindowsDialog extends Dialog {
    Button createButton() { return new WindowsButton(); }
}
```

Abstract Factory example:

```
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
class WinFactory implements GUIFactory {
    public Button createButton() { return new WinButton(); }
    public Checkbox createCheckbox() { return new WinCheckbox(); }
}
```

When to use which:

- Use **Factory Method** when you just need to create **one kind of object**, but let subclasses pick the exact variant.
 - Use **Abstract Factory** when you need to create **several related objects that must be consistent with each other** (e.g., all UI widgets in the same visual theme).
-

Q41. What is the motivation of the Abstract Factory pattern? What are the participant objects? Briefly explain the role of each. Show example code demonstrating its use. (asked 3x)

A: Motivation: When a system needs to create **families of related objects** and must guarantee they're **compatible with each other** — without the client code needing to know the concrete classes involved (and without hardcoding which family is being used).

Participants:

- **AbstractFactory** — declares creation methods for each type of product (interface).
- **ConcreteFactory** — implements those methods, creating a specific family of products.
- **AbstractProduct** — declares an interface for a type of product object.
- **ConcreteProduct** — the actual product implementation, created by a matching ConcreteFactory.
- **Client** — uses only the AbstractFactory and AbstractProduct interfaces, never touching concrete classes directly.

Example code:

```
interface Button { void render(); }
interface Checkbox { void render(); }

class MacButton implements Button { public void render() { System.out.println("Mac button"); } }
class MacCheckbox implements Checkbox { public void render() { System.out.println("Mac checkbox"); } }

interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class MacFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

class Application {
    private GUIFactory factory;
    Application(GUIFactory factory) { this.factory = factory; }
    void render() {
        factory.createButton().render();
        factory.createCheckbox().render();
    }
}
```

The `Application` never mentions `MacButton` directly — it only knows `GUIFactory`, `Button`, `Checkbox`.

Q42. Meal ordering system: main dish (mandatory), one or more sides, optional drink, optional dessert — how would you design this for flexible, readable, step-by-step/chained construction? Demonstrate with a UML class diagram.

A: This is a classic **Builder pattern** scenario — object construction is complex, has optional parts, and needs to be readable.

Why Builder fits: A constructor with 4+ optional parameters becomes unreadable ("telescoping constructor" smell). Builder lets you build the object **step by step**, calling only the parts you need, in a fluent/chained style.

```

class Meal {
    private String mainDish;
    private List<String> sides = new ArrayList<>();
    private String drink;
    private String dessert;
    // private constructor, only Builder can build it
    private Meal() {}

    static class Builder {
        private Meal meal = new Meal();
        Builder mainDish(String dish) { meal.mainDish = dish; return this; }
        Builder addSide(String side) { meal.sides.add(side); return this; }
        Builder drink(String drink) { meal.drink = drink; return this; }
        Builder dessert(String dessert) { meal.dessert = dessert; return this; }
        Meal build() {
            if (meal.mainDish == null) throw new IllegalStateException("Main dish required");
            return meal;
        }
    }
}

// Usage - reads like natural language:
Meal meal = new Meal.Builder()
    .mainDish("Burger")
    .addSide("Fries")
    .addSide("Salad")
    .drink("Coke")
    .build(); // dessert skipped - optional

```

UML (described):

```

Meal

- mainDish, sides, drink, dessert
+ static Builder

Meal.Builder

- meal: Meal
+ mainDish(String): Builder
+ addSide(String): Builder
+ drink(String): Builder
+ dessert(String): Builder
+ build(): Meal

```

Each **Builder** method returns **this**, enabling **method chaining**. Mandatory fields (main dish) can be enforced in **build()** or the Builder constructor; optional ones simply aren't called.

Q43. Briefly describe the differences between the Builder pattern and the Abstract Factory Pattern.

A: | | Builder | Abstract Factory | |---|---|---| | Goal | Construct **one complex object step-by-step** | Create **families of simple, related objects** in one call | | Construction | Multi-step, incremental (**.setX().setY().build()**) | Single method call per product (**createButton()**) | | Focus | **How** an object is built (the process) | **What** related objects are produced together | | Result | Usually returns **one** finished object at the end | Returns **multiple different objects** meant to be used together | | Client involvement | Client can control the building steps/order | Client just asks for a product; no control over how it's built internally |

In short: Builder is about **constructing one thing carefully**, Abstract Factory is about **producing a matching set of things quickly**.

Q44. Purchase request approval system — Manager approves ≤ 10,000, Director approves > 10,000. Design using an appropriate pattern; draw class diagram and write client code. Then: Assistant Director added (Manager ≤10,000, Assistant Director ≤100,000, Director >100,000) — show minimal changes needed. *(asked multiple times, both with & without Assistant Director tier)*

A: This is the classic **Chain of Responsibility** pattern — a request travels along a chain of handlers, and each either handles it or passes it forward.

Why not just if/else? Because we want to be able to **add/remove/reorder approvers without touching existing code** — exactly the flexibility the change request (adding Assistant Director) tests.

```
abstract class Approver {
    protected Approver next;
    void setNext(Approver next) { this.next = next; }
    abstract void processRequest(double amount);
}

class Manager extends Approver {
    void processRequest(double amount) {
        if (amount <= 10000) {
            System.out.println("Manager approved: " + amount);
        } else if (next != null) {
            next.processRequest(amount);
        }
    }
}

class Director extends Approver {
    void processRequest(double amount) {
        if (amount > 10000) {
            System.out.println("Director approved: " + amount);
        } else if (next != null) {
            next.processRequest(amount);
        }
    }
}

// Client code
Approver manager = new Manager();
Approver director = new Director();
manager.setNext(director);
manager.processRequest(15000); // passes to Director
```

UML (described):

```
Approver (abstract)

- next: Approver
+ setNext (Approver)
+ processRequest (double)
  ^
  |
Manager      Director      AssistantDirector
```

Minimal change for Assistant Director tier:

1. Add a new class `AssistantDirector extends Approver` that approves when `10000 < amount <= 100000`.
2. Change `Director` 's condition to `amount > 100000`.
3. Update the **chain wiring** in client code:

```
Approver manager = new Manager();
Approver assistantDirector = new AssistantDirector();
Approver director = new Director();
manager.setNext(assistantDirector);
assistantDirector.setNext(director);
```

No existing `Manager / Director` **class structure** needed to change (only the Director's threshold and the chain wiring) — proving the design is extensible, per **Open/Closed Principle**.

Q45. Travel agency tour packages: hierarchical structure with activities and nested sub-packages (accommodation → hotel/resort, transportation → rental car/airport shuttle). Suggest a pattern and draw a UML diagram.

A: This is the classic **Composite** pattern — a tree structure of "whole-part" objects that should be treated uniformly, whether they're a single leaf item or a whole nested group.

Why: A tour package can *contain* other packages/activities, recursively, to any depth — the client code should be able to treat a single "Sightseeing" activity and a whole "Accommodation" sub-package (with hotel + resort inside) **the same way**.

```
interface PackageComponent {
    void showDetails();
}

class Activity implements PackageComponent { // Leaf
    String name;
    public void showDetails() { System.out.println(name); }
}

class TourPackage implements PackageComponent { // Composite
    String name;
    List<PackageComponent> children = new ArrayList<>();
    void add(PackageComponent c) { children.add(c); }
    public void showDetails() {
        System.out.println(name + ":");
        for (PackageComponent c : children) c.showDetails();
    }
}

// Usage
TourPackage accommodation = new TourPackage(); accommodation.name = "Accommodation";
accommodation.add(new Activity() {{ name = "Hotel"; }});
accommodation.add(new Activity() {{ name = "Resort"; }});

TourPackage mainTour = new TourPackage(); mainTour.name = "Full Tour";
mainTour.add(new Activity() {{ name = "Sightseeing"; }});
mainTour.add(accommodation); // nested composite
```

UML (described):

```
PackageComponent (interface)
+ showDetails()
    ^
    |
Activity          TourPackage
(leaf)            - children: List<PackageComponent>
                  + add(PackageComponent)
                  (composite — contains other PackageComponents, including itself recursively)
```

Q46. File system: files and folders — a folder contains files and/or other folders; add/remove folder/file; list files. Design the system: class diagram + code. *(asked 2x)*

A: Again **Composite** pattern — Files (leaves) and Folders (composites containing Files/Folders) must be handled uniformly.

```
interface FileSystemItem {
    String getName();
    int getSize();
}

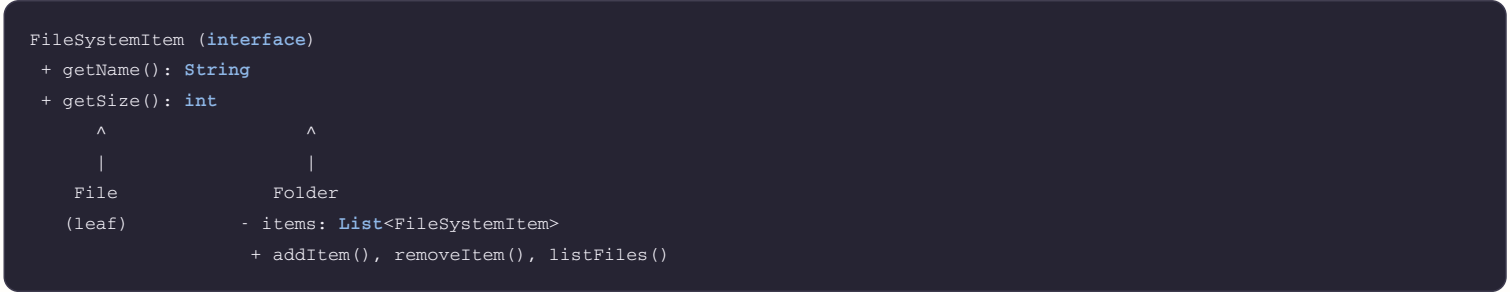
class File implements FileSystemItem { // Leaf
    String fileName, fileExtension, location;
    Date creationTime;
    int fileSize;
    public String getName() { return fileName + "." + fileExtension; }
    public int getSize() { return fileSize; }
}

class Folder implements FileSystemItem { // Composite
    String folderName, location;
    Date creationTime;
    List<FileSystemItem> items = new ArrayList<>();

    void addItem(FileSystemItem item) { items.add(item); }
    void removeItem(FileSystemItem item) { items.remove(item); }
    List<FileSystemItem> listFiles() { return items; }

    public String getName() { return folderName; }
    public int getSize() {
        int total = 0;
        for (FileSystemItem item : items) total += item.getSize();
        return total; // recursively sums nested folder sizes too
    }
}
```

UML (described):



Because `Folder.getSize()` calls `getSize()` on each child — which could itself be a `Folder` — sizes are computed **recursively**, which is the core power of Composite.

Q47. Computer game with African/Asian/American scenarios, each with a terrain, 5 trees of same kind, 2 animals of same kind — different terrain/tree/animal kind per scenario. Design and write code. *(asked 2x)*

A: This is **Abstract Factory** — each scenario is a **family of related objects** (terrain + tree + animal) that must all match the same "theme" (region), and the client shouldn't need to know which concrete classes it's creating.

```
interface Terrain { void render(); }
interface Tree { void render(); }
interface Animal { void render(); }

// African family
class AfricanTerrain implements Terrain { public void render() { System.out.println("Savannah"); } }
class AfricanTree implements Tree { public void render() { System.out.println("Acacia tree"); } }
class AfricanAnimal implements Animal { public void render() { System.out.println("Lion"); } }

interface ScenarioFactory {
    Terrain createTerrain();
    Tree createTree();
    Animal createAnimal();
}

class AfricanScenarioFactory implements ScenarioFactory {
    public Terrain createTerrain() { return new AfricanTerrain(); }
    public Tree createTree() { return new AfricanTree(); }
    public Animal createAnimal() { return new AfricanAnimal(); }
}

class Scenario {
    void build(ScenarioFactory factory) {
        factory.createTerrain().render();
        for (int i = 0; i < 5; i++) factory.createTree().render();
        for (int i = 0; i < 2; i++) factory.createAnimal().render();
    }
}
```

Swapping `AfricanScenarioFactory` for `AsianScenarioFactory` changes the **entire family** consistently, with zero changes to `Scenario`.

Q48. Auction shop: auctioneer accepts bids; bidder raises paddle; bid price changes; all other bidders must be notified; auctioneer broadcasts new bid. Design, class diagram + code.

A: This is the **Observer** pattern — many `Bidder` objects need to be **automatically notified** whenever the auction's state (current bid) changes, without the `Auctioneer` needing to know bidder details individually.

```

interface Bidder { // Observer
    void update(double newBid);
}

class ConcreteBidder implements Bidder {
    String name;
    public void update(double newBid) {
        System.out.println(name + " notified: new bid is " + newBid);
    }
}

class Auctioneer { // Subject
    private List<Bidder> bidders = new ArrayList<>();
    private double currentBid;

    void registerBidder(Bidder b) { bidders.add(b); }
    void removeBidder(Bidder b) { bidders.remove(b); }

    void acceptBid(double amount) {
        currentBid = amount;
        notifyAllBidders();
    }

    private void notifyAllBidders() {
        for (Bidder b : bidders) b.update(currentBid);
    }
}

```

UML (described):

Auctioneer (Subject)	Bidder (Observer interface)
- bidders: List<Bidder>	+ update(double)
- currentBid: double	^
+ registerBidder(Bidder)	
+ acceptBid(double)	ConcreteBidder
+ notifyAllBidders()	-----> (calls update on each)

SECTION 5: Structural Patterns

Q49. What patterns seem similar to the Proxy pattern? Explain why they are similar. What are the dissimilarities that made these patterns exist separately? *(asked 2x)*

A: Similar patterns: Decorator and **Adapter** (and sometimes compared to **Facade**).

Why similar to Decorator: Both Proxy and Decorator **wrap another object** and implement the same interface as the object they wrap, forwarding calls to it. Structurally, their UML diagrams look almost identical.

Why similar to Adapter: Both Proxy and Adapter wrap an object and expose an interface — but for Adapter, the wrapped object has a **different** interface that needs converting, while Proxy keeps the **same** interface.

Dissimilarities (why they exist as separate patterns): | | Proxy | Decorator | Adapter | |---|---|---|---| | Purpose | **Controls access** to the real object (lazy loading, caching, access control, logging) | **Adds new behavior/responsibility** to an object dynamically | **Converts one interface into another** so incompatible classes can work together | | Same interface as wrapped object? | Yes, identical | Yes, identical | No — different interface | | Knows about wrapped object at creation? | Usually manages/creates it internally (often lazily) | Client explicitly wraps an object it already has | Wraps an existing incompatible object to translate calls |

In short: Decorator is about **adding**, Proxy is about **controlling/guarding** access, Adapter is about **translating** interfaces — same "wrapper" shape, different intent.

Q50. What patterns are similar to the Composite pattern? Why are they similar? What are the differences that make them exist separately? *(asked 2x)*

A: Similar pattern: Decorator.

Why similar: Both Composite and Decorator rely on **recursive composition** — an object can contain other objects of the same abstract type, and both use a tree-like/recursive structure where operations cascade through nested objects.

Differences: | | Composite | Decorator | |---|---|---| | Purpose | Represents **part-whole hierarchies** — treats a single object and a group of objects **uniformly** | **Adds extra responsibilities/behavior** to a single object dynamically | | Number of children | Can have **many** children (0 to many) | Usually wraps exactly **one** component | | Goal | Structural organization of objects | Enhancing/extending one object's behavior |

In short: Composite is about **grouping many into a tree**; Decorator is about **layering behavior onto one object**. Both use recursive wrapping, but for different reasons.

Q51. What patterns are similar to the Decorator pattern? Why similar? What are the differences?

A: *(Same core comparisons as above, from Decorator's point of view.)*

Similar to: Proxy, Composite, Chain of Responsibility — all these patterns wrap objects and forward requests through a chain/tree of similarly-typed objects (structurally nearly identical, as noted by design pattern literature — even the *chain-of-responsibility pattern is structurally nearly identical to the decorator pattern*, the key difference being how many objects in the chain actually handle each request).

Pattern	How it differs from Decorator
Proxy	Controls access instead of adding behavior; usually one proxy, not stackable layers
Composite	Organizes a tree of many children , not a single wrapped object
Chain of Responsibility	Passes a request along a chain where exactly one handler typically processes it and stops, while in Decorator every wrapper in the chain contributes to the behavior

Decorator's unique trait: It's designed to be **stacked** — you can wrap a wrapped object again and again, adding new independent behaviors each time, all while keeping the same interface as the original object.

Q52. Image loading mechanism: `RealImage` loads from disk in its constructor and displays via `display()` . What are the problems with this? Which pattern(s) solve it? Show UML.

A: Problem: `RealImage` loads the file from disk **immediately when constructed** (`loadFromDisk()` runs in the constructor), even if `display()` is never called. This wastes memory/time for images that might never actually be shown — no **lazy loading**.

Pattern: Proxy (specifically a **Virtual Proxy**) — a lightweight placeholder object that defers the expensive loading until it's actually needed.

```
interface Image { void display(); }

class RealImage implements Image {
    String fileName;
    RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk(fileName); // heavy operation
    }
    public void display() { System.out.println("Displaying " + fileName); }
    private void loadFromDisk(String f) { System.out.println("Loading " + f); }
}

class ProxyImage implements Image { // Virtual Proxy
    private String fileName;
    private RealImage realImage; // not created yet!
    ProxyImage(String fileName) { this.fileName = fileName; }
    public void display() {
        if (realImage == null) { // lazy load, only when needed
            realImage = new RealImage(fileName);
        }
        realImage.display();
    }
}
```

UML (described):

```
Image (interface)
+ display()
    ^
    |
RealImage        ProxyImage
(loads on        - realImage: RealImage (lazy)
construction)    + display() → creates RealImage only when called
```

Q53. When `RealImage` fails to load, `SimpleImage` is tried; if that fails, `ImageContent` loads a textual description. Which pattern fits this? Show UML.

A: This is **Chain of Responsibility** — a request (load the image) is passed along a chain of handlers (`RealImage` → `SimpleImage` → `ImageContent`) until one of them successfully "handles" it (loads).

```
abstract class ImageHandler {
    protected ImageHandler next;
    void setNext(ImageHandler next) { this.next = next; }
    abstract boolean tryLoad(String path);
}

class RealImageHandler extends ImageHandler {
    boolean tryLoad(String path) {
        if (canLoadReal(path)) { /* load and display */ return true; }
        return next != null && next.tryLoad(path);
    }
    boolean canLoadReal(String path) { return false; /* placeholder */ }
}

class SimpleImageHandler extends ImageHandler {
    boolean tryLoad(String path) {
        if (canLoadSimple(path)) { /* load simple image */ return true; }
        return next != null && next.tryLoad(path);
    }
    boolean canLoadSimple(String path) { return false; }
}

class ImageContentHandler extends ImageHandler {
    boolean tryLoad(String path) {
        System.out.println("Loading text description for " + path);
        return true; // always succeeds — end of chain
    }
}
```

UML (described):

```
ImageHandler (abstract)

- next: ImageHandler
+ tryLoad(path): boolean
  ^
  |
RealImageHandler → SimpleImageHandler → ImageContentHandler
(chain wired in this order via setNext)
```

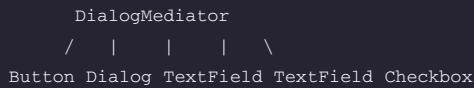
Q54. UI dependency diagram (Button ↔ Dialog ↔ TextField ↔ TextField ↔ Checkbox, all cross-connected) — what are the problems? Which pattern(s) solve it? Redraw the diagram. *(asked 2x, same exact diagram)*

A: Problem: Every UI component **directly references and depends on several other components** — this is a **tangled web of many-to-many dependencies**. Consequences:

- **Tight coupling** — changing one component (e.g., `Checkbox`) risks breaking several others that reference it directly.
- **Hard to reuse** any single component elsewhere, since it drags its neighbors with it.
- **Hard to understand/maintain** — there's no single place that manages the interaction logic; it's scattered across all components.

Pattern: Mediator — introduce a central object that all components talk to **instead of talking to each other directly**. Components only know about the Mediator, not each other.

Redrawn structure (described):



All arrows now point **only to/from the Mediator** — no more direct lines between `Button` and `Checkbox`, etc.

```
interface Mediator {
    void notify(Component sender, String event);
}

class DialogMediator implements Mediator {
    Button button; Checkbox checkbox; TextField field1, field2;
    public void notify(Component sender, String event) {
        if (sender == checkbox && event.equals("checked")) {
            field1.enable();
        }
        // ... central place for all cross-component logic
    }
}

abstract class Component {
    protected Mediator mediator;
    Component(Mediator m) { this.mediator = m; }
}

class Button extends Component {
    Button(Mediator m) { super(m); }
    void click() { mediator.notify(this, "click"); }
}
```

Now every component only holds a reference to the **Mediator**, and all cross-component logic lives in one central, manageable place.

Q55. "Strategy changes the guts of an object whereas Decorator changes the skins of an object" — explain this statement with appropriate reasoning. (asked 3x)

A: This is a classic memory phrase distinguishing two patterns that both "swap in" behavior, but at different levels:

- **Strategy changes the "guts"** — it replaces the **internal algorithm/logic** an object uses to do a task, while the object's outer identity and interface stay the same. Example: a `SortContext` can be given `QuickSort` or `MergeSort` — the *inner working* changes, but from outside, the object is still "a sorter."

```
class SortContext {  
    private SortStrategy strategy; // 0 — swappable algorithm  
    void setStrategy(SortStrategy s) { strategy = s; }  
    void sort(int[] arr) { strategy.sort(arr); }  
}
```

- **Decorator changes the "skin"** — it **wraps** an object from the **outside**, adding new behavior **around** the existing object without touching its internal logic at all. The object's core "guts" never change; extra layers are added on top, like layers of clothing.

```
Coffee coffee = new MilkDecorator(new SugarDecorator(new BasicCoffee()));  
// BasicCoffee's own logic never changes — behavior is layered around it
```

In short: Strategy = swap the **engine inside**; Decorator = add **layers outside**. Both achieve flexible behavior, but Strategy changes *what algorithm runs*, while Decorator changes *what extra stuff happens around* the same core behavior.

Q65. Which design pattern is used to hide the complexity of a complex subsystem? Draw the UML/class diagram, and identify the participants and their roles.

A: The **Facade** pattern.

Purpose: Provide a **single, simplified interface** to a complex subsystem made up of many interacting classes, so client code doesn't need to understand or interact with all of them directly.

Participants:

- **Facade** — the simplified front-facing class the client talks to; it delegates work to the subsystem classes.
- **Subsystem classes** — the complex internal classes that do the real work; they don't know about the Facade and can be used directly if needed.
- **Client** — uses only the Facade, never touching subsystem classes directly (though it *could*).

```
// Subsystem classes
class CPU { void freeze() {} void jump(long pos) {} void execute() {} }
class Memory { void load(long pos, byte[] data) {} }
class HardDrive { byte[] read(long lba, int size) { return new byte[size]; } }

// Facade
class ComputerFacade {
    private CPU cpu = new CPU();
    private Memory memory = new Memory();
    private HardDrive hardDrive = new HardDrive();

    void start() { // one simple method hides a lot of complexity
        cpu.freeze();
        memory.load(0, hardDrive.read(0, 1024));
        cpu.jump(0);
        cpu.execute();
    }
}

// Client
new ComputerFacade().start(); // client never touches CPU/Memory/HardDrive directly
```

UML (described):



SECTION 6: Behavioral Patterns

Q56. Notification system: one developer suggests Observer (User = Observer, event sources = Subject); another suggests Mediator (central NotificationMediator). Critically compare both for scalability (new event types), flexibility (new delivery channels), and performance bottlenecks. Which do you recommend and why?

A:

Approach 1 — Observer (`User` observes `Post` /event-source `Subject` s):

- ✓ **Simple**, direct, well-understood — each event source just notifies its subscribed users.
- ✗ **Scalability problem:** every new event type (e.g., "event reminders") means creating a **new Subject type** and wiring up new observer relationships — this grows messy fast with many event types.
- ✗ **Flexibility problem:** adding new delivery channels (SMS, Slack) means every single `User.update()` implementation must handle branching logic for each channel — channel logic ends up duplicated across every event/observer pairing.
- ✗ **Performance risk:** if many event sources each maintain their own observer lists, and a user is subscribed everywhere, notification fan-out logic gets scattered and harder to optimize/batch.

Approach 2 — Mediator (`NotificationMediator` routes everything):

- ✓ **Scalability:** new event types just register with the central mediator — no new relationships to wire between individual objects.
- ✓ **Flexibility:** delivery channel logic (email/push/SMS/Slack) lives in **one place** — the mediator can loop through a user's enabled channels for any event type, so adding a channel means updating the mediator once, not every observer.
- ✗ **Performance risk:** the mediator can become a **bottleneck** / "**God object**" if not carefully designed, since all traffic flows through it.

Recommendation: Mediator, because the requirements explicitly say the business plans to **keep growing** both event types and delivery channels — Mediator centralizes that growth in one maintainable place, while Observer would scatter it across many small classes (violating DRY as channels multiply). Observer alone doesn't scale well when *combinations* of (event type × channel) grow, which is exactly this scenario.

(A hybrid is also valid: use Observer for the "a Post has followers" relationship, but route the actual channel-dispatch logic through a Mediator — showing you understand both patterns can co-exist.)

Q57. Same notification system: avoid large conditional logic when checking a user's channel preferences per event — propose a design that dynamically builds a "pipeline" of notification actions. Which pattern(s)?

A: Instead of:

```
if (user.wantsEmail()) sendEmail();
if (user.wantsPush()) sendPush();
if (user.wantsSms()) sendSms();
```

use the **Chain of Responsibility** pattern to build a dynamic pipeline of "delivery handlers," combined with **Strategy** for each individual channel's sending logic.

```
interface NotificationHandler {
    void setNext(NotificationHandler next);
    void handle(Notification n, User user);
}

class EmailHandler implements NotificationHandler {
    private NotificationHandler next;
    public void setNext(NotificationHandler next) { this.next = next; }
    public void handle(Notification n, User user) {
        if (user.wantsEmail()) sendEmail(n, user);
        if (next != null) next.handle(n, user);
    }
    void sendEmail(Notification n, User user) { /* ... */ }
}

// similarly PushHandler, SmsHandler, SlackHandler...

// Build pipeline dynamically based on what's registered
NotificationHandler pipeline = new EmailHandler();
pipeline.setNext(new PushHandler());
// Adding SMS later = new SmsHandler(), one line — no existing code touched
```

Unlike classic Chain of Responsibility (where only ONE handler acts), here **every enabled handler in the chain contributes** — closer to a **"pipeline"** — but the wiring/extensibility benefit is the same: **new channels = new handler classes**, and the "pipeline" of handlers is assembled dynamically at runtime instead of a hardcoded if-chain. (**Strategy** can additionally be used inside each handler if the *sending mechanism itself* varies.)

Q58. Explain how the Command pattern decouples the "invoker" from the "receiver." Draw the UML class diagram.

A: Command turns a request/action into a **standalone object** that carries all the information needed to perform an action later. This means:

- The **Invoker** (who triggers the action, e.g., a button or `UserInterface`) only needs to call `command.execute()` — it never needs to know which class actually performs the work, or which method to call.
- The **Receiver** (the object that actually does the work, e.g., `TextEditor`) is only referenced *inside* the Command object, not by the Invoker.

```
interface Command { void execute(); }

class SaveCommand implements Command {
    private TextEditor editor; // Receiver
    SaveCommand(TextEditor editor) { this.editor = editor; }
    public void execute() { editor.save(); }
}

class PrintCommand implements Command {
    private TextEditor editor;
    PrintCommand(TextEditor editor) { this.editor = editor; }
    public void execute() { editor.print(); }
}

class UserInterface { // Invoker
    private Command saveCommand, printCommand;
    UserInterface(Command save, Command print) {
        this.saveCommand = save; this.printCommand = print;
    }
    void simulateSaveClick() { saveCommand.execute(); }
    void simulatePrintClick() { printCommand.execute(); }
}
```

UML (described):

```
Invoker (UserInterface)  -->  Command (interface)
                             ^
                             |
                        SaveCommand  PrintCommand

                        - editor: TextEditor (Receiver)
                        + execute()
                          |
                          v
                        TextEditor (Receiver)
```

Now `UserInterface` calls `execute()` on any `Command` — it doesn't know or care that behind the scenes it's `editor.save()` .

Q59. After refactoring to Command pattern, how could you implement a command history to support an "undo" operation?

A: Because each Command is now its own object, you can **store executed commands in a list (a history stack)** and add an `undo()` method to reverse each one.

```
interface Command {
    void execute();
    void undo();
}

class SaveCommand implements Command {
    private TextEditor editor;
    private String previousState; // remember state before executing
    public void execute() {
        previousState = editor.getContent();
        editor.save();
    }
    public void undo() {
        editor.restore(previousState);
    }
}

class CommandHistory { // keeps a stack of executed commands
    private Deque<Command> history = new ArrayDeque<>();
    void push(Command c) { history.push(c); }
    void undoLast() {
        if (!history.isEmpty()) {
            history.pop().undo();
        }
    }
}

class UserInterface {
    private CommandHistory history = new CommandHistory();
    void executeCommand(Command c) {
        c.execute();
        history.push(c);
    }
    void undo() { history.undoLast(); }
}
```

Each command, before executing, saves whatever state is needed to reverse itself. The `CommandHistory` is just a stack — undo pops the most recent command and calls its `undo()`.

Q60. What is the motivation of the Command pattern? Draw the UML/class diagram for Command pattern. Identify the participants and their roles from the diagram.

A: Motivation: To **turn a request into a standalone object**, so requests can be:

- **Queued** (stored and executed later),
- **Logged** (for undo/redo or auditing),
- **Parameterized** (pass an action as a method argument, like a callback),
- and to **decouple** the object that invokes an action from the object that knows how to perform it.

Participants:

- **Command** (interface) — declares `execute()` (and often `undo()`).
- **ConcreteCommand** — implements `execute()` by calling one or more methods on a Receiver; stores whatever state is needed.
- **Receiver** — the class that actually knows how to perform the operation.
- **Invoker** — holds a Command and triggers `execute()` , with no knowledge of what the command actually does.
- **Client** — creates ConcreteCommand objects and configures them with the correct Receiver, then hands them to the Invoker.

UML:

```
Client --creates--> ConcreteCommand --executes on--> Receiver
      ^
      |
      Command (interface: execute(), undo())
      ^
      |
Invoker --- holds a --- Command
+ setCommand(Command)
+ executeCommand()
```

Q61. What are the participant objects in the Decorator pattern? Explain the role of each. Show example code that demonstrates the use of Decorator pattern.

A: Participants:

- **Component** (interface) — declares the common interface for both wrapped and wrapping objects.
- **ConcreteComponent** — the original/base object being decorated.
- **Decorator** (abstract) — implements `Component` and holds a reference to a wrapped `Component`, delegating calls to it.
- **ConcreteDecorator** — adds actual new behavior before/after delegating to the wrapped component.

```
interface Coffee { double cost(); String description(); }

class BasicCoffee implements Coffee { // ConcreteComponent
    public double cost() { return 2.0; }
    public String description() { return "Coffee"; }
}

abstract class CoffeeDecorator implements Coffee { // Decorator
    protected Coffee wrapped;
    CoffeeDecorator(Coffee c) { this.wrapped = c; }
}

class MilkDecorator extends CoffeeDecorator { // ConcreteDecorator
    MilkDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 0.5; }
    public String description() { return wrapped.description() + " + Milk"; }
}

class SugarDecorator extends CoffeeDecorator { // ConcreteDecorator
    SugarDecorator(Coffee c) { super(c); }
    public double cost() { return wrapped.cost() + 0.2; }
    public String description() { return wrapped.description() + " + Sugar"; }
}

// Usage — stack decorators freely at runtime
Coffee order = new SugarDecorator(new MilkDecorator(new BasicCoffee()));
System.out.println(order.description() + " = $" + order.cost());
// 4
```

Q62. What is the motivation of the Strategy pattern? What are the participant objects in the Observer pattern? Draw the UML/class diagram for Command pattern. Briefly explain the role of each participant object.

A: (This exam question combines three sub-parts.)

Motivation of Strategy: To define a **family of interchangeable algorithms**, encapsulate each one in its own class, and make them swappable at runtime — so the object using the algorithm doesn't need `if/else` / `switch` to pick behavior, and new algorithms can be added without changing existing code (Open/Closed Principle).

Participants of Observer:

- **Subject** — holds a list of Observers; provides `attach()`, `detach()`, `notify()`.
- **ConcreteSubject** — stores the state that Observers care about; calls `notify()` when that state changes.
- **Observer** (interface) — declares `update()`, the method Subject calls.
- **ConcreteObserver** — implements `update()` to react to the Subject's change (e.g., refresh a UI, log something).

Command pattern UML + roles: (see Q60 for the full diagram — *Command*, *ConcreteCommand*, *Receiver*, *Invoker*, *Client*.)

Q63. Name two different patterns that are used to eliminate if-else statements in a code. Give an example for each to explain how these patterns help eliminate if-else. *(asked 2x)*

A: 1. Strategy pattern — replaces `if/else` branching on **which algorithm to run** with polymorphism:

```
// Before (if-else)
double calculateShipping(String type, double weight) {
    if (type.equals("standard")) return weight * 1.0;
    else if (type.equals("express")) return weight * 2.5;
    else return weight * 5.0;
}

// After (Strategy)
interface ShippingStrategy { double calculate(double weight); }
class StandardShipping implements ShippingStrategy {
    public double calculate(double weight) { return weight * 1.0; }
}
class ExpressShipping implements ShippingStrategy {
    public double calculate(double weight) { return weight * 2.5; }
}
// Client picks a strategy object instead of branching
```

2. State pattern — replaces `if/else` branching on **an object's current state** with polymorphism, where each state is its own class:

```
// Before (if-else on state)
void handle() {
    if (state.equals("OPEN")) { /* open logic */ }
    else if (state.equals("CLOSED")) { /* closed logic */ }
}

// After (State pattern)
interface DoorState { void handle(); }
class OpenState implements DoorState {
    public void handle() { /* open logic */ }
}
class ClosedState implements DoorState {
    public void handle() { /* closed logic */ }
}
// The Door object just calls currentState.handle() — no branching
```

(Chain of Responsibility, Factory Method / Replace Type Code with Subclasses are also valid alternative answers.)

Q66. Which design pattern helps removing duplication in common workflow? Draw the UML/class diagram for the pattern. Identify the participants and their roles from the diagram. (asked 2x)

A: The **Template Method** pattern.

How it removes duplication: When several classes follow the **same overall algorithm/steps**, but a few steps differ between them, Template Method puts the **shared skeleton in a base class** (as a single, non-overridable method), and lets subclasses override only the **specific steps that vary**.

```
abstract class ReportGenerator { // AbstractClass
    // Template Method — the fixed skeleton, same for everyone
    final void generateReport() {
        fetchData();
        formatData();
        exportReport();
    }
    void fetchData() { System.out.println("Fetching data..."); } // shared default
    abstract void formatData(); // must vary per subclass
    void exportReport() { System.out.println("Exporting report..."); } // shared default
}

class PdfReportGenerator extends ReportGenerator { // ConcreteClass
    void formatData() { System.out.println("Formatting as PDF"); }
}

class CsvReportGenerator extends ReportGenerator { // ConcreteClass
    void formatData() { System.out.println("Formatting as CSV"); }
}
```

Participants:

- **AbstractClass** — defines the `templateMethod()` (fixed skeleton) and declares abstract "hook" steps for subclasses to fill in; may also provide default implementations for some steps.
- **ConcreteClass** — implements only the varying steps; inherits the shared skeleton unchanged.

UML (described):

```
ReportGenerator (AbstractClass)
+ generateReport() [final — the template]
+ fetchData() [default]
# formatData() [abstract — must override]
+ exportReport() [default]
    ^
    |
PdfReportGenerator      CsvReportGenerator
# formatData() override  # formatData() override
```

This directly removes **Duplicate Code** across classes that share the same overall process but differ in a few steps.

SECTION 7: Memory Tricks 🧠

Use these to lock in the patterns fast before the exam.

🕒 The Three Families

"Birth, Body, Brain"

- **Creational = Birth** (how objects are *born*)
- **Structural = Body** (how objects are *built together*)
- **Behavioral = Brain** (how objects *think/talk* to each other)

🔧 Creational Patterns — "SOFA-B" style memory hook

Think of furnishing a new house — **S**ingle owner, **O**ne factory, **F**amily factory, **A**ssemble step by step, **B**e a clone:

- **Singleton** → only **one** instance (one key to one house)
- Factory **M**ethod → one method decides **which** product subclass to make
- **Abstract Factory** → a **factory of factories** — makes a whole matching **family** (like ordering a full furniture set that all matches)
- **Builder** → build **step by step**, like assembling IKEA furniture with instructions
- **Prototype** → **clone** an existing object instead of building from scratch (like photocopying a form)

🏠 Structural Patterns — "Wrap it, Bridge it, Fake it"

- **Adapter** = **Translator** (plug converter — different interface, same job)
- **Decorator** = **Add toppings** (pizza with extra cheese — same pizza, more layers)
- **Proxy** = **Bodyguard/Gatekeeper** (controls access, may delay the real thing)
- **Facade** = **TV remote** (one button hides a ton of complex wiring behind the TV)
- **Composite** = **Folder tree** (a folder can hold files AND folders — treated the same way)
- **Bridge** = **Remote + Device** (separate the "what" from the "how", so both can vary independently)
- **Flyweight** = **Shared crayon box** (many students share the same box instead of each owning one)

Proxy vs Decorator vs Adapter memory line: "*Adapter translates. Proxy guards. Decorator decorates.*" All three wrap — but only Decorator wants to **stack**, only Proxy wants to **control**, only Adapter wants to **translate**.

🗣️ Behavioral Patterns — "Who's in charge of the conversation?"

- **Observer** = **Newsletter subscription** (subscribers get notified automatically when something changes)
- **Strategy** = **Choose your travel mode** (walk, drive, fly — same goal, different "engine" inside)
- **Command** = **Restaurant order slip** (the waiter/invoke doesn't cook — the slip carries everything the chef/receiver needs)
- **Template Method** = **Recipe with blanks** (fixed steps, but some steps you fill in yourself)
- **Chain of Responsibility** = **Customer support escalation** (Level 1 → Level 2 → Manager, whoever can handle it, does)
- **Mediator** = **Air traffic control tower** (planes don't talk to each other directly, only to the tower)
- **State** = **Traffic light** (its behavior changes completely depending on current state: red/yellow/green)
- **Iterator** = **TV remote's channel-up button** (steps through a collection one at a time, without exposing how it's stored)

📌 Code Smell memory groups (Fowler's categories, as used by Refactoring Guru)

"BOCDC" – **B**loaters, **OO** Abusers, **C**hange Preventers, **D**ispensables, **C**ouplers

- **Bloaters** = things that grew **too big**: Long Method, Large Class, Long Parameter List, Primitive Obsession, Data Clumps
- **OO Abusers** = **misusing OOP** principles: Switch Statements, Refused Bequest, Temporary Field, Alternative Classes with Different Interfaces
- **Change Preventers** = one change forces edits **everywhere**: Divergent Change, Shotgun Surgery, Parallel Inheritance Hierarchies
- **Dispensables** = **unnecessary** clutter: Comments, Duplicate Code, Lazy Class, Speculative Generality, Dead Code
- **Couplers** = **too tightly connected**: Feature Envy, Inappropriate Intimacy, Message Chains, Middle Man

🔍 "Which pattern removes which smell?" — quick lookup

Code Smell	Refactor Toward
Switch on type code	Replace Conditional with Polymorphism → Strategy / State / Factory Method
Null checks everywhere	Null Object pattern
Object creation is complex/optional fields	Builder
Need multiple related objects created together	Abstract Factory
Direct tight coupling between invoker & action	Command
Many objects wired directly to each other	Mediator
Same algorithm, different steps in subclasses	Template Method
Need to add behavior without touching original class	Decorator
Too many similar heavy objects (memory issue)	Flyweight
Sequential fallback / escalation logic	Chain of Responsibility
Need to control/delay access to a heavy object	Proxy
Whole-part tree structure	Composite

📖 One-liner intents (say these out loud 3x before the exam)

1. **Singleton** — only one instance, globally accessible.
2. **Factory Method** — let subclasses decide what to create.
3. **Abstract Factory** — create families of related objects.
4. **Builder** — build complex objects step by step.
5. **Prototype** — clone existing objects.
6. **Adapter** — make incompatible interfaces work together.
7. **Bridge** — separate abstraction from implementation.
8. **Composite** — treat individual & group objects uniformly (tree).
9. **Decorator** — add behavior dynamically by wrapping.
10. **Facade** — one simple interface for a complex subsystem.
11. **Flyweight** — share common data to save memory.
12. **Proxy** — control access to another object.
13. **Chain of Responsibility** — pass request along a chain of handlers.
14. **Command** — turn a request into an object.
15. **Iterator** — traverse a collection without exposing its structure.
16. **Mediator** — centralize communication between objects.
17. **Observer** — notify dependents automatically when state changes.
18. **State** — change behavior when internal state changes.
19. **Strategy** — swap interchangeable algorithms at runtime.
20. **Template Method** — fixed skeleton, subclasses fill in steps.

SECTION 8: Final Cheat Sheet 📄

A. All 23 GoF Patterns at a Glance

#	Pattern	Type	One-line Intent
1	Singleton	Creational	One instance, global access
2	Factory Method	Creational	Subclass decides what to create
3	Abstract Factory	Creational	Create matching families of objects
4	Builder	Creational	Build complex object step by step
5	Prototype	Creational	Clone existing objects

6	Adapter	Structural	Convert one interface to another
7	Bridge	Structural	Decouple abstraction from implementation
8	Composite	Structural	Tree of part-whole objects, treated uniformly
9	Decorator	Structural	Add behavior dynamically via wrapping
10	Facade	Structural	Simplify access to a complex subsystem
11	Flyweight	Structural	Share data to save memory
12	Proxy	Structural	Control access to another object
13	Chain of Responsibility	Behavioral	Pass request along a chain of handlers
14	Command	Behavioral	Encapsulate a request as an object
15	Iterator	Behavioral	Traverse a collection uniformly
16	Mediator	Behavioral	Centralize object communication
17	Memento	Behavioral	Capture and restore object state
18	Observer	Behavioral	Notify dependents on state change
19	State	Behavioral	Change behavior based on internal state
20	Strategy	Behavioral	Swap interchangeable algorithms
21	Template Method	Behavioral	Fixed skeleton, subclasses fill steps
22	Visitor	Behavioral	Add new operations without changing classes
23	Interpreter	Behavioral	Interpret sentences in a language

B. Code Smell Quick Reference

Smell	Symptom	Fix
Long Method	Method does too much	Extract Method
Large Class	Class has too many responsibilities	Extract Class
Long Parameter List	Too many params passed around	Introduce Parameter Object
Duplicate Code	Same logic copy-pasted	Extract Method / Pull Up Method
Switch Statements	Repeated type-code branching	Replace Conditional with Polymorphism
Feature Envy	Method more interested in another class's data	Move Method
Data Clumps	Same group of fields/params everywhere	Extract Class / Parameter Object
Primitive Obsession	Using raw types for domain concepts	Replace Data Value with Object
Shotgun Surgery	One change needs edits in many classes	Move Method / Inline Class
Divergent Change	One class changed for many different reasons	Extract Class
Refused Bequest	Subclass doesn't want inherited stuff	Push Down Method / Replace Inheritance with Delegation
Lazy Class	Class barely does anything	Inline Class / Collapse Hierarchy
Speculative Generality	Unused "just in case" flexibility	Collapse Hierarchy / Remove abstraction
Message Chains	<code>a.getB().getC().getD()</code>	Hide Delegate
Middle Man	Class only delegates, does nothing else	Remove Middle Man
Comments (as deodorant)	Explaining bad code instead of fixing it	Extract Method + Rename
Parallel Inheritance Hierarchies	Every subclass needs a matching subclass elsewhere	Move Method to merge hierarchies

C. "If you see THIS code shape → THIS pattern" Rapid Recognition

Code Shape / Scenario Keyword	Pattern to Suggest
"only one instance should exist"	Singleton
"family of related objects", "matching theme/set"	Abstract Factory

"optional fields", "step-by-step construction"	Builder
"clone/copy an object"	Prototype
"different interface, needs to fit"	Adapter
"tree structure", "folder contains files and folders"	Composite
"add feature without modifying original class"	Decorator
"hide complex subsystem behind one entry point"	Facade
"too many similar objects eating memory"	Flyweight
"control/delay access to expensive object"	Proxy
"escalation", "fallback chain", "try X then Y then Z"	Chain of Responsibility
"undo/redo", "queue actions", "decouple invoker from receiver"	Command
"loop through collection without exposing internals"	Iterator
"many objects wired to each other directly"	Mediator
"save/restore state (undo snapshot)"	Memento
"notify subscribers on change"	Observer
"behavior changes completely based on status"	State
"swap algorithm at runtime"	Strategy
"same steps, different sub-steps"	Template Method

D. Answering-strategy tips for the exam

- When asked **"which pattern... why... draw UML"** — always structure your answer as: **(1) Problem/Motivation → (2) Pattern name → (3) Participants & roles → (4) UML/code → (5) Why it solves the problem.**
- When asked to **identify code smells**, name **the smell explicitly** (not just "this is bad code") — examiners look for the exact term (e.g., "Primitive Obsession," not "using too many strings").
- When asked to **refactor**, name **the refactoring technique** you're using (e.g., "Extract Method," "Replace Conditional with Polymorphism") — don't just show new code silently.
- For **"similar patterns" questions** (Proxy vs Decorator vs Adapter, Composite vs Decorator) — always give a **table**: similarity first, then the **one key differentiator**.
- For **UML diagrams**, always show: interface/abstract class at top, concrete implementations below with inheritance arrows, and any "has-a" (composition) relationships as a separate arrow with a label.
- If a question combines OOP theory + pattern (e.g., "why OOP", "interface vs abstract class") — anchor your answer back to **flexibility, reusability, and reduced coupling** — these three words appear in almost every "why" answer in this subject.

E. Last-minute revision order (highest exam frequency first)

- Comments as deodorant (*asked 4x*)
- Validator/ValidatorFactory refactor exercise (*asked 4x*)
- Primitive Obsession (*asked 4x*)
- Feature Envy (*asked 3x*)
- Collapse Hierarchy (*asked 3x*)
- Refused Bequest (*asked 3x*)
- Strategy vs Decorator ("guts vs skins") (*asked 3x*)
- Can interface be replaced by abstract class? (*asked 3x*)
- Purchase approval system → Chain of Responsibility (*asked 3x*)
- Abstract Factory motivation & example (*asked 3x*)